SEM5 — Software Engineering & Project Management

Practical Assignment 1 (Revised & Evidence-Verified)

Software Development Process Analysis for a Real-World Application:

GlobalMedX — Pandemic Surveillance & Response Platform

SDLC Analysis, Model Selection, Risk Assessment, and Verified Implementation Status

Own project: github.com/adarsh985/globalmedx — verified directly against source code

Submitted by
Adarsh Singh
Roll No: 150096724164
Semester 5 — Software Engineering & Project Management

Note: This revision distinguishes Implemented, Configured, Documented, Planned, Unverified, and Missing items. Configuration files are not treated as proof of successful execution, and proposed Agile/Scrum activities are not presented as historical evidence.

1. Problem Statement

GlobalMedX is a full-stack pandemic-surveillance and response web application, originally built by the author as a project for a Semester 4 DevOps course and reused here as the subject system for SDLC analysis. It has a React (Vite) frontend and a Node.js/Express/MongoDB backend, plus a set of DevOps configuration assets (Docker, Kubernetes manifests, a Jenkins pipeline, Terraform scripts, Prometheus/Grafana configs, an ELK logging pipeline, and a HashiCorp Vault setup). It is a solo student project, not a production enterprise system, and this report is careful to say so wherever a claim cannot be independently verified from the code. This report analyses the SDLC that was realistically followed (and that should be followed) to build and evolve it, and selects, justifies, and compares SDLC models suited to its actual scale and maturity.

Verification method: the GitHub repository was downloaded and inspected directly — source files, package.json dependencies, the Jenkinsfile, Kubernetes YAML, Terraform files, and Vault/Prometheus config — rather than relying only on the README's description.

2. SDLC Concept & Generic Phases

Phase	Purpose
Planning	Define project scope, feasibility, and timeline.
Requirement Analysis	Gather what the system must do (functional) and how well (non-functional).
Design	Translate requirements into architecture, data models, and UI/UX.
Implementation	Write and integrate the code based on the design.
Testing	Verify the system meets requirements and is defect-free.
Deployment	Release the system to users.
Maintenance	Fix defects, patch, and add enhancements post-release.

3. Stakeholder Identification

Stakeholder	Role / Interest
Public Health Officials / Epidemiologists	Would view outbreak and case-trend data to inform decisions (simulated data in this project, not real health data).
Hospitals / Healthcare Providers	Would report case and bed-capacity data via the Hospital and Report modules.
Border/Airport Authorities	Would use the Airport/Border module to log traveller screening.
Laboratory Staff	Would log diagnostic test results via the Laboratory module.
System Administrator	Uses the Admin Portal and DevOps dashboard to monitor the platform.
Developer (the author)	Designs, builds, and maintains all layers of the stack solo.
Course Evaluator	Assesses the SDLC process, documentation, and engineering practice.

4. Requirement Analysis

Every item below is tagged with its verified evidence level: Implemented means real, working code exists for it; Configured means it exists as infrastructure/config only; Unverified means it cannot be confirmed without running the live system.

4.1 Functional Requirements (verified against backend controllers & frontend components)

- Users can register and log in; passwords are hashed with bcrypt and sessions use JWT tokens (authController.js, User.js). **[Implemented]**
- Role-based route protection restricts actions by user role (authMiddleware.js: protect, authorize). **[Implemented]**
- A global dashboard aggregates case/death/recovery figures (dashboardController.js, GlobalDashboard.jsx). **[Implemented]**
- Disease/outbreak case reports can be created and queried (reportController.js, Report.js, Outbreak.js, Surveillance.jsx). **[Implemented]**
- Hospitals and bed capacity can be registered and tracked (hospitalController.js, Hospital.js, HospitalManager.jsx). **[Implemented]**
- Laboratory diagnostic results can be logged (labController.js, Laboratory.js, Laboratory.jsx). **[Implemented]**
- Airport/border traveller screening can be logged (airportController.js, Airport.js, AirportBorder.jsx). **[Implemented]**
- Analytics charts render trend data using Chart.js (analyticsController.js, Analytics.jsx). **[Implemented]**
- Alerts/incidents can be raised and tracked (alertController.js, Alert.js, Incident.js, EmergencyResponse.jsx). **[Implemented]**
- An outbreak simulator can generate/mutate sample data for demo purposes (simulationController.js, Simulator.jsx). **[Implemented]**
- An admin portal shows system-level info and user/credential management (adminController.js, AdminPortal.jsx). **[Implemented]**
- A DevOps dashboard displays live CPU/memory/API telemetry pulled from a Prometheus client in the backend (devopsController.js, prometheus.js, DevopsDashboard.jsx). **[Implemented]**
- Backend secrets (DB URI, JWT secret) can be fetched from HashiCorp Vault at boot, with automatic fallback to environment variables if Vault is unreachable (config/vault.js). **[Implemented in code]**

4.2 Non-Functional Requirements

- Security: passwords are hashed (bcrypt) and API routes use JWT + RBAC middleware. **[Implemented]**
- Observability: the backend is instrumented with a Prometheus client exposing request-count and latency metrics. **[Implemented]**
- Centralised logging: Winston structured logging exists in the backend; a Logstash pipeline config exists to ship logs to Elasticsearch. **[Partially implemented / Configured]**
- Scalability: a Kubernetes Horizontal Pod Autoscaler manifest is defined for the backend deployment. **[Configured, not load-tested]**
- Infrastructure reproducibility: Terraform scripts define an AWS VPC and EKS cluster. **[Configured, not verified applied]**
- Automated CI/CD: a Jenkinsfile defines a 7-stage pipeline (checkout, install, test, build, docker build, push, deploy). **[Present, but test/push/deploy stages are mocked (see Section 12)]**
- Automated test coverage: no unit, integration, or end-to-end tests exist anywhere in the repository. **[Missing]**

5. SDLC Model Selection & Justification

An Agile-DevOps hybrid is recommended as the target SDLC for GlobalMedX. The architecture supports its principles through modular feature development, containerisation, infrastructure-as-code, a CI/CD pipeline

definition, and monitoring configurations. However, the repository evidence supports describing this as a target approach rather than a fully executed Agile-Scrum/DevOps process.

However, in the interest of honesty, the GitHub history for this repository currently shows a small number of commits by a single author, with no branches, pull requests, issues, or sprint boards visible — so while the model recommended below is well justified by the architecture, it should not be presented as a Scrum process that was actually run with ceremonies, sprints, or a team. What actually happened is closer to iterative solo development: features were added and refined incrementally, which is compatible with Agile's spirit even without formal Scrum artifacts.

5.1 Why Agile-DevOps fits the architecture

- The system is split into independent functional modules (auth, dashboard, hospital, lab, airport, analytics, alerts, simulator, admin, DevOps) that could be developed and released as separate increments.
- Containerisation (Docker) and Kubernetes manifests are designed for frequent, repeatable, independent deployment of each service — a DevOps prerequisite.
- The Jenkinsfile models the intended automation flow of build, test, package, and deploy; it does not yet demonstrate complete automated delivery because the test, image-push, and deployment steps are placeholders.
- Monitoring (Prometheus/Grafana) and logging (ELK) configs exist to close the feedback loop that Agile-DevOps depends on.

5.2 Honest limitation of this evidence

- No branches, pull requests, or issue tracking were found — so a team-based Scrum workflow cannot be claimed.
- The pipeline's test and deployment stages are explicitly mocked, so continuous delivery has not actually been demonstrated end-to-end.
- Recommendation for future work: adopt real sprints (even solo, using a personal Kanban/issue board), write actual tests, and un-mock the Jenkins stages to genuinely practise Agile-DevOps rather than only modelling it.

6. SDLC Lifecycle Diagram

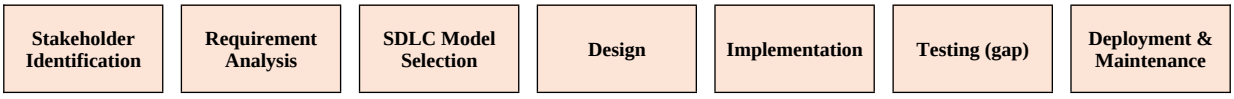


Figure 1: SDLC lifecycle as applicable to GlobalMedX. The Testing phase is marked as a gap because no automated tests currently exist (see Section 11).

7. Identification of Possible Software Risks

Risk	Likelihood	Impact	Mitigation Strategy
No automated tests: regressions can ship silently	High	High	Add unit tests (Jest) for controllers and integration tests for API routes
Jenkins pipeline stages are mocked, giving false confidence in CI/CD	High	Medium	Un-mock real 'npm test' and real docker push/kubectl apply once tests exist
Hard-coded default secrets in code as fallback (e.g. default JWT secret string)	Medium	High	Require Vault or environment secrets in all non-local environments

Risk	Likelihood	Impact	Mitigation Strategy
No verified live deployment; Vercel/Render targets configured but unconfirmed	Medium	Medium	Confirm and document actual deployment status before claiming production readiness
Single point of failure: solo development, no code review	Medium	Medium	Use pull requests even solo, and add a linting/CI gate before merging to main
MongoDB or Vault unavailable at startup	Low	Medium	Existing fallback-to-env-vars logic mitigates the Vault case; add DB retry/backoff

8. Software Failure Case Analysis

This section discusses a general, industry-known failure pattern relevant to health-data dashboards under load. It is a hypothetical scenario for learning purposes, not a documented incident that occurred on this project (no incident logs or monitoring history were found for GlobalMedX itself).

8.1 Generic Root Causes (industry pattern, not project-specific)

- Underestimated peak load during capacity planning for a sudden traffic surge.
- Tight coupling between an ingestion API and a dashboard read-path, causing cascading slowdowns.
- Deploying without an automated rollback plan.

8.2 How this applies to GlobalMedX specifically

- Because there is no test suite, a regression could reach the (would-be) production system undetected.
- Because the Jenkins deploy stage is mocked, there is currently no evidence the described Kubernetes rollout/rollback would actually work if triggered.
- Lesson: verify the pipeline end-to-end in a real (even single-node) cluster before relying on it for any real release.

9. Comparison with Alternative SDLC Models

Criteria	Waterfall	Spiral	Agile-DevOps (Recommended)
Flexibility to change	Low — costly once signed off	Moderate — iteration allows adjustment	High — modules can change per iteration
Risk handling	Addressed late, often at testing	Explicit risk review each loop	Should be surfaced via monitoring and tests (not yet fully realised here)
Suited to this project's current maturity	Overkill for a solo, evolving project	Useful for the DevOps risk-heavy parts (Terraform/K8s)	Best long-term fit once testing and CI are real
Delivery speed	Single delivery at the end	Incremental with prototypes	Continuous delivery: currently only modelled, not executed

9.1 Waterfall Model (reference — recreate in Draw.io)

Requirement Analysis

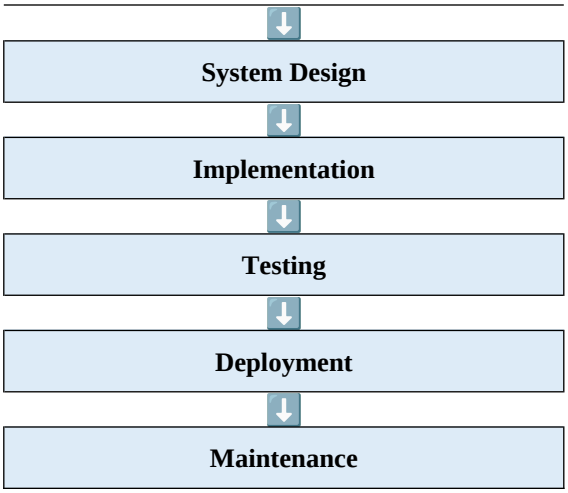


Figure 2: Waterfall Model — sequential phases. Recreate this in Draw.io for submission (see chat instructions).

9.2 Spiral Model (reference — recreate in Draw.io)

Quadrant	Activity
1. Determine Objectives	Define goals, alternatives, and constraints for this iteration.
2. Identify & Resolve Risks	Analyse risks; prototype to resolve high-risk items (e.g. the untested CI/CD pipeline).
3. Develop & Test	Develop the increment and verify it against requirements.
4. Plan Next Iteration	Review and plan the next spiral loop.

Figure 3: Spiral Model — four repeating quadrants, spiralling outward each iteration.

10. SDLC Applied to a Simpler System — ATM (for Lucidchart)

As required by the assignment, the same lifecycle approach is applied to a simple, well-defined, unrelated system: an ATM. Waterfall fits better here because requirements are fixed and safety-critical. This is intentionally kept separate from GlobalMedX. Full Lucidchart reproduction instructions are given in the chat response.

- Requirement Analysis: identify transaction types (withdraw, deposit, balance enquiry, PIN change).
- System Design: define hardware-software interaction, security protocol, and database schema.
- Implementation: develop transaction-processing and card-authentication modules.
- Testing: validate transaction accuracy, security, and failure handling (e.g. power loss mid-transaction).
- Deployment: install and configure ATMs at branch locations.
- Maintenance: cash replenishment, software patches, hardware servicing.

11. Testing Artifacts — Current Status

Verified finding: the repository contains no test files of any kind (no *.test.js, no __tests__ folder, no Jest/Mocha/Cypress dependency in either package.json), and the Jenkinsfile's test stage is a placeholder echo statement, explicitly commented "Mocked tests for college presentation." Testing is therefore a genuine gap, not an oversight in this report.

11.1 Test Case Template (to be executed and filled in by the student)

Test Case ID	Feature	Preconditions	Steps	Expected Result	Status
TC-01	User login	A user exists in MongoDB with a known password	POST /api/auth/login with valid credentials	200 response with a JWT token	**Not Executed**
TC-02	Invalid login	Same as above	POST /api/auth/login with wrong password	401 response, no token issued	**Not Executed**
TC-03	Role-restricted route	A non-admin JWT is available	Call an admin-only route with that token	403 Forbidden	**Not Executed**
TC-04	Create case report	A valid JWT is available	POST a new report via the Surveillance module	201 response; report appears in dashboard totals	**Not Executed**
TC-05	Vault fallback	VAULT_TOKEN is unset	Start the backend without Vault running	Backend starts using env-variable secrets, logs a fallback warning	**Not Executed**

11.2 Bug Report Template

Bug ID	Description	Steps to Reproduce	Severity	Status
BUG-01	No verified bugs currently logged; fill in once tests above are executed	N/A	N/A	Not Verified

12. GitHub Repository & Version Control — Verified Status

- Repository: github.com/adarsh985/globalmedx, with real folders /frontend, /backend, /docker, /k8s, /terraform, /jenkins, /monitoring, /elk, /vault, /docs, all confirmed to contain real, non-empty files.
- Commit history is small (single-digit commit count) on a single author's timeline, consistent with solo, incremental development rather than a multi-branch team workflow.
- The README describes a branching strategy (main / develop / feature-*) and a commit-message convention. These are documented practices, not verified historical evidence, because the visible repository history does not confirm use of the additional branches. The README should also avoid describing CI/CD as complete while the test, push, and deployment stages remain placeholders.
- The Jenkinsfile defines a realistic 7-stage pipeline shape, but the test, docker-push, and kubectl-apply steps are commented out and replaced with echo statements, so the committed pipeline does not perform real automated testing or deployment yet.
- No .github/workflows directory exists, so there is no GitHub Actions-based CI in addition to the Jenkinsfile.

13. Project Planning (labelled Planned, not historical)

Because this project was built solo without formal role assignment, the WBS below is a retrospective, planned structure for how the work breaks down, not a record of an actual multi-person schedule tracked in real time.

13.1 Work Breakdown Structure (Planned)

WBS ID	Work Package	Status
1.0	Requirement gathering & stakeholder analysis	**Planned**
2.0	SDLC & DevOps pipeline planning	**Planned**
3.0	Frontend UI development (11 React components)	**Implemented**
4.0	Backend & API development (10 controllers, 8 Mongoose models)	**Implemented**
5.0	Containerisation & IaC (Docker, Terraform, K8s manifests)	**Configured**
6.0	CI/CD pipeline definition (Jenkins)	**Configured**
7.0	Observability stack (Prometheus, Grafana, ELK)	**Configured**
8.0	Secrets management (Vault)	**Implemented**
9.0	Testing (unit, integration, load)	**Missing**
10.0	Deployment & release management	**Configured**
11.0	Documentation & reporting	**Implemented**

13.2 Suggested Forward-Looking Gantt (by Week)

Activity	Wk 1-2	Wk 3-4	Wk 5-6	Wk 7-8
Write unit & integration tests	X	X		
Un-mock Jenkins pipeline stages		X	X	
Verify live deployment (Vercel/Render)		X		
Load-test Kubernetes HPA behaviour			X	X
Document actual results in this report				X

13.3 Sample Sprint Backlog (proposed, not yet executed as Scrum)

User Story	Priority	Story Points	Sprint
As a developer, I want unit tests for authController so login/register logic is verified	High	5	Sprint 1
As a developer, I want the Jenkins test stage to run real 'npm test' instead of an echo	High	3	Sprint 1
As a developer, I want to confirm the Vercel/Render deployment actually serves the app	Medium	3	Sprint 1
As a developer, I want the Jenkins deploy stage to apply real k8s manifests to a test cluster	Medium	8	Sprint 2

14. UML & Architecture Diagrams — Status and Specification

No UML diagrams currently exist in the repository. The specifications below are what should be created (e.g. in draw.io or a UML tool) to satisfy the assignment's UML/architecture requirement.

14.1 Use Case Diagram (to be created)

- Actors: Health Official, Hospital Staff, Lab Staff, Border/Airport Staff, Admin.
- Use cases: Login, View Dashboard, Submit Case Report, Manage Hospital Capacity, Log Lab Result, Log Border Screening, View Analytics, Raise Alert, Run Simulation, Manage Users (Admin).

14.2 Class Diagram (to be created, based on the 8 real Mongoose models)

- Classes: User, Report, Outbreak, Hospital, Laboratory, Airport, Alert, Incident, Resource, with the fields already defined in backend/src/models/*.js.

14.3 Sequence Diagram (to be created)

- Suggested flow: Login sequence. Frontend to POST /api/auth/login to authController to User model (bcrypt.compare) to JWT issued to Frontend stores token.

14.4 System Architecture Diagram (to be created)

- Tiers: React SPA to Express API (with auth/logging middleware) to MongoDB; side integrations: Vault (secrets), Prometheus (metrics), Logstash/Elasticsearch (logs), matching docs/project_report.md's architecture sketch already in the repo.

15. Implementation Verification Summary

Consolidated evidence classification for the whole project, for transparency and for the evaluator.

Area	Status	Evidence
Frontend (React/Vite, 11 components)	**Implemented**	Real .jsx source, ~3,500 lines, wired to backend via axios
Backend (Express/Mongoose, 10 controllers, 8 models)	**Implemented**	Real route handlers and schemas in backend/src/
Authentication (JWT + bcrypt + RBAC)	**Implemented**	authController.js, authMiddleware.js
Metrics instrumentation (Prometheus client)	**Implemented**	config/prometheus.js wired into index.js
Logging (Winston)	**Implemented**	middleware/loggingMiddleware.js
Docker / docker-compose	**Configured**	Dockerfiles and compose file present; run status not confirmed in this review
Kubernetes manifests	**Configured**	8 YAML files present; kubectl apply is commented out in Jenkinsfile
Terraform (AWS VPC/EKS)	**Configured**	6 .tf files present; no state file or apply evidence
HashiCorp Vault integration	**Implemented**	vault.js has real fetch logic with fallback; live Vault use not confirmed
ELK logging pipeline	**Configured**	logstash.conf present; no evidence of ingestion having run
Jenkins CI/CD	**Configured**	Jenkinsfile present; test/push/deploy steps are echo placeholders
Automated tests	**Missing**	No test files or test dependencies anywhere in the repo

Area	Status	Evidence
Live deployment (Vercel/Render)	**Documented**	vercel.json and README link to it; reachability not confirmed in this review
Documentation	**Implemented**	7 guides in docs/ (deployment, monitoring, jenkins, kubernetes, terraform, disaster recovery, project report)

16. Conclusion

GlobalMedX contains implemented frontend and backend functionality, supported by a DevOps configuration layer. Its modular architecture makes Agile-DevOps a reasonable target SDLC. Nevertheless, the evidence shows that the process has not been demonstrated end-to-end: no automated test suite was found, key Jenkins stages are placeholders, and live deployment and observability execution remain unverified. Therefore, this report presents Agile-DevOps as the recommended target model while clearly separating implemented code, configured infrastructure, documented plans, and missing verification.